

# LegalEase

## AI-Powered Legal Document Generator

*Complete Project Documentation + Coding Examples*

Based on the supplied LegalEase project documentation and Skill Wallet screenshots

# 1. Project Overview

LegalEase is an AI-powered legal document generator. The supplied project documentation describes a Streamlit frontend, FastAPI backend, Gemini-based generation, editable preview, and TXT/DOCX/PDF export. The user enters a document type, parties, terms and effective date; the application generates structured legal content for review and editing.

# 2. Objectives

- Generate structured legal document drafts from user inputs.
- Provide editable previews.
- Export documents as TXT, DOCX and PDF in the documented full implementation.
- Connect Streamlit to FastAPI through an API endpoint.
- Integrate Gemini for AI-assisted generation.
- Test the complete workflow locally before deployment.

# 3. Technologies

| Layer         | Technology        | Role                               |
|---------------|-------------------|------------------------------------|
| Frontend      | Streamlit         | Input, preview, editing, downloads |
| Backend       | FastAPI + Uvicorn | API and application logic          |
| AI            | Google Gemini     | Document generation                |
| Validation    | Pydantic          | Request validation                 |
| Word          | python-docx       | DOCX export                        |
| PDF           | FPDF              | PDF export                         |
| Images        | Pillow            | Logo/image handling                |
| HTTP          | Requests          | Frontend-to-backend communication  |
| Configuration | python-dotenv     | Environment variables              |

# 4. Architecture

User → Streamlit → FastAPI/generate → Gemini → Generated Draft → Edit/Preview → TXT/DOCX/PDF

1. Streamlit collects document\_type, parties, terms and effective\_date.
2. FastAPI validates and receives the request.
3. GeminiDocumentGenerator creates the AI response.
4. Formatting utilities convert the response into downloadable formats.
5. Streamlit displays and allows editing of the result.

# 5. Prerequisites and Setup

- 6. Install Python 3.10+ and pip.
- 7. Create a project folder.
- 8. Create a virtual environment.
- 9. Install project dependencies.
- 10. Create .env for configuration.
- 11. Create the backend and frontend folders.
- 12. Add the Python source files.

## Create virtual environment

```
python -m venv .venv
```

## Install documented dependencies

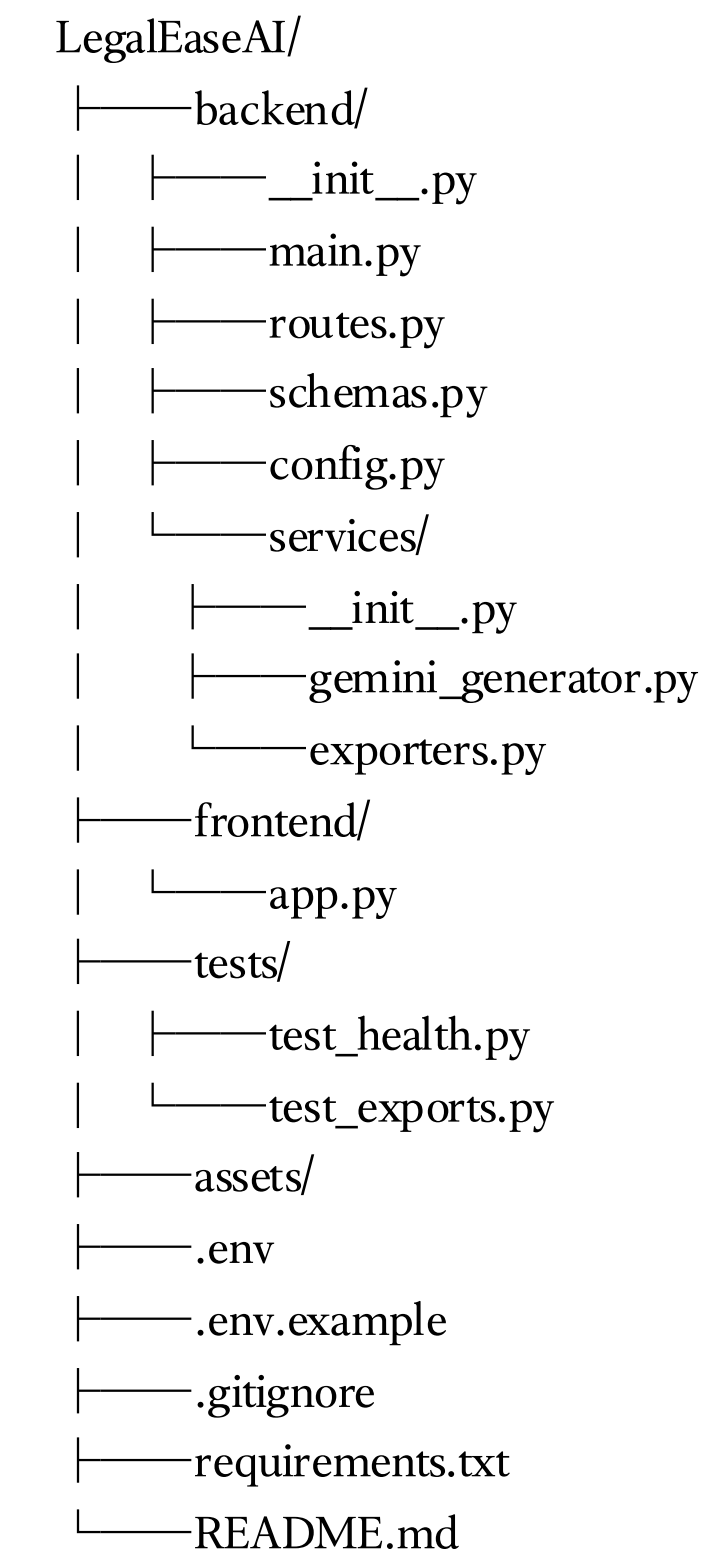
```
pip install fastapi uvicorn streamlit python-docx fpdf Pillow requests google-generativeai python-dotenv
```

## Run current Windows venv Python without PowerShell activation

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

# 6. Recommended Project Structure

## Folder structure



# 7. Requirements File

## requirements.txt

```
fastapi
uvicorn[standard]
streamlit
requests
python-dotenv
pydantic
pydantic-settings
google-genai
python-docx
fpdf2
Pillow
pytest
httpx
```

## 8. Environment Configuration

### **.env**example

```
APP_NAME=LegalEase
BACKEND_HOST=127.0.0.1
BACKEND_PORT=8000
FRONTEND_BACKEND_URL=http://127.0.0.1:8000
GEMINI_API_KEY=
GEMINI_MODEL=gemini-2.5-flash
DEMO_MODE=true
```

Keep API keys private. Do not commit a real key to a public repository.

## 9. Milestone 1– Model Selection and Architecture

### 9.1 Model Selection

The supplied project documentation identifies Gemini 1.5 Pro as the selected model for the documented implementation. The current local working configuration may use a different Gemini model name if the environment is configured that way.

- Understand the document-generation requirement.
- Review model context and structured generation capabilities.
- Select a model suitable for formal document drafting.
- Keep model configuration in the environment rather than hard-coding secrets.

### 9.2 Architecture Configuration Code

#### **backend/config.py**

```
import os
from dotenv import load_dotenv

load_dotenv()

APP_NAME=os.getenv("APP_NAME", "LegalEase")
BACKEND_HOST=os.getenv("BACKEND_HOST", "127.0.0.1")
```

```
BACKEND_PORT=int(os.getenv("BACKEND_PORT", "8000"))
FRONTEND_BACKEND_URL=os.getenv(
    "FRONTEND_BACKEND_URL", "http://127.0.0.1:8000"
)
GEMINI_API_KEY=os.getenv("GEMINI_API_KEY", "")
GEMINI_MODEL=os.getenv("GEMINI_MODEL", "gemini-2.5-flash")
DEMO_MODE=os.getenv("DEMO_MODE", "true").lower() == "true"
```

## 10. Milestone 2– Core Functionality

### 10.1 Request Schema

**backend/schemas.py**

```
from pydantic import BaseModel

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    effective_date: str

class GenerateResponse(BaseModel):
    document_type: str
    text: str
    terms: list[str]
```

### 10.2 Core Document Generation Example

**backend/routes.py– simple working/demo generation**

```
from fastapi import APIRouter
from backend.schemas import DocumentRequest, GenerateResponse

router = APIRouter()

@router.get("/health")
def health():
    return {"status": "ok", "service": "LegalEase"}

@router.post("/generate", response_model=GenerateResponse)
def generate_document(request: DocumentRequest):
    terms_list = [
        item.strip()
        for item in request.terms.replace(";", "\n").splitlines()
        if item.strip()
    ]

    terms_text = "\n".join(f"- {term}" for term in terms_list)
    if not terms_text:
```

```
terms_text= "[NOT PROVIDED]"
```

```
document=f"""{request.document_type.upper()}
```

```
EFFECTIVE DATE
{request.effective_date}
```

```
PARTIES
{request.parties}
```

```
PURPOSE
This document records the terms and conditions agreed by the parties.
```

```
KEY TERMS
{terms_text}
```

```
GENERAL PROVISIONS
This document is a draft for review. Missing information should be completed before signing.
"""
```

```
return GenerateResponse(
    document_type=request.document_type,
    text=document,
    terms=terms_list
)
```

### 10.3 Gemini Generator Example

This example illustrates the role of the Gemini service described in the project documentation. The exact SDK syntax can vary with the installed Google SDK version, so keep the SDK version and API documentation aligned with the environment.

**backend/services/gemini\_generator.py – illustrative service structure**

```
import os

class GeminiDocumentGenerator:
    def __init__(self):
        self.api_key = os.getenv("GEMINI_API_KEY", "")
        self.model = os.getenv("GEMINI_MODEL", "gemini-2.5-flash")

    def build_prompt(self, document_type, parties, terms, effective_date):
        return f"""
Create a structured legal document draft.

Document type: {document_type}
Parties: {parties}
Terms: {terms}
Effective date: {effective_date}

Use clear sections and preserve the supplied facts.
Return a draft for review.
"""
```

```

def generate_document(
    self, document_type, parties, terms, effective_date
):
    prompt = self.build_prompt(
        document_type, parties, terms, effective_date
    )

    # Connect the prompt to the selected Gemini SDK/model here.
    # Return the model-generated text.
    return prompt

```

## 10.4 Exporter Examples

### backend/services/exporters.py– TXT example

```

def build_txt(text: str) -> bytes:
    return text.encode("utf-8")

```

### DOCX export concept

```

from docx import Document

def build_docx(text: str) -> bytes:
    doc = Document()
    for line in text.splitlines():
        doc.add_paragraph(line)
    # Save to an in-memory buffer in the full implementation.
    return b""

```

### PDF export concept

```

from fpdf import FPDF

def build_pdf(text: str) -> bytes:
    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Helvetica", size=11)
    for line in text.splitlines():
        pdf.multi_cell(0, 7, line)
    # Export to an in-memory buffer in the full implementation.
    return pdf.output()

```

The examples above demonstrate the roles of the export functions; production code should return the correctly constructed byte streams for FastAPI downloads.

## 11. Milestone 3– API Logic Integration

### 11.1 FastAPI main.py

backend/main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from backend.routes import router

app = FastAPI(
    title="LegalEase",
    description="AI Legal Document Generator",
    version="1.0.0"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"]
)

app.include_router(router)

@app.get("/")
def root():
    return {
        "message": "LegalEase API is running.",
        "docs": "/docs",
        "health": "/health"
    }
```

11.2 API Endpoints

| Method | Endpoint  | Purpose           |
|--------|-----------|-------------------|
| GET    | /         | API status        |
| GET    | /health   | Health check      |
| POST   | /generate | Generate document |

Example JSON request

```
{
  "document_type": "Rental Agreement",
  "parties": "Landlord: Arun Kumar; Tenant: Swetha",
  "terms": "Monthly rent is Rs. 15000; Security deposit is Rs. 30000",
  "effective_date": "01-10-2026"
}
```

12. Milestone 4 – Frontend Development

## 12.1 Streamlit Frontend Example

**frontend/app.py**– compact working version

```
import os
import requests
import streamlit as st
from dotenv import load_dotenv

load_dotenv()

BACKEND_URL = os.getenv(
    "FRONTEND_BACKEND_URL",
    "http://127.0.0.1:8000"
)

st.set_page_config(
    page_title="LegalEase",
    page_icon="⚖️",
    layout="centered"
)

st.title("⚖️ LegalEase")
st.subheader("AI Legal Document Generator")

document_type = st.text_input(
    "Document Type (Ex: Agreement, Contract, NDA)"
)
parties = st.text_area("Parties Involved")
terms = st.text_area(
    "Terms & Conditions (Use semicolons for bullet points)"
)
effective_date = st.text_input("Effective Date")

if st.button("Generate Document"):
    if not all([
        document_type.strip(),
        parties.strip(),
        terms.strip(),
        effective_date.strip()
    ]):
        st.error("Please complete all fields.")
    else:
        response = requests.post(
            f"{BACKEND_URL}/generate",
            json={
                "document_type": document_type,
                "parties": parties,
                "terms": terms,
                "effective_date": effective_date
            },
            timeout=60
        )
```

```

if response.status_code == 200:
    result = response.json()
    edited = st.text_area(
        "Edit Document",
        value=result["text"],
        height=500
    )
    st.download_button(
        "Download TXT",
        data=edited,
        file_name="LegalEase_Document.txt",
        mime="text/plain"
    )
else:
    st.error(response.text)

```

## 12.2 Frontend-to-Backend Flow

13. User fills the four input fields.
14. Streamlit creates a JSON request.
15. Request sends POST/generate to FastAPI.
16. FastAPI validates the JSON with Pydantic.
17. The generation service produces document text.
18. FastAPI returns JSON.
19. Streamlit displays the document.
20. User edits and downloads the result.

# 13. Milestone 5– Deployment

## 13.1 Start Backend

### Terminal 1

```
.\.venv\Scripts\python.exe -m uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000
```

Expected: Uvicorn reports that the application is running on <http://127.0.0.1:8000>.

## 13.2 Start Frontend

### Terminal 2

```
.\.venv\Scripts\python.exe -m streamlit run frontend/app.py
```

Expected: Streamlit opens the app, normally at <http://localhost:8501>.

## 13.3 API Verification

### Open API documentation

<http://127.0.0.1:8000/docs>

### Open health endpoint

http://127.0.0.1:8000/health

# 14. Testing and Verification

## 14.1 Sample Input

| Field          | Sample                                                                                                                                                                                   |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Document Type  | Rental Agreement                                                                                                                                                                         |
| Parties        | Landlord: Arun Kumar<br>Tenant: Swetha<br>Property: Chennai, Tamil Nadu                                                                                                                  |
| Terms          | Monthly rent is Rs. 15000; Security deposit is Rs. 30000; Rent must be paid before the 5th of every month; The tenant will keep the property clean; The agreement is valid for 11 months |
| Effective Date | 01-10-2026                                                                                                                                                                               |

- 21. Open `http://localhost:8501`.
- 22. Enter the sample values.
- 23. Click Generate Document.
- 24. Confirm the generated document appears.
- 25. Check the parties, terms and effective date.
- 26. Edit the result.
- 27. Download the output.
- 28. Open the downloaded file and verify it.
- 29. Repeat using another document type and different terms.

## 14.2 Automated Test Examples

`tests/test_health.py`

```
from fastapi.testclient import TestClient
from backend.main import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"
```

### Run tests

```
.\.venv\Scripts\python.exe -m pytest
```

# 15. Skill Wallet Task Board – Status Separation

The following is based on the three supplied screenshots. Truncated titles remain truncated.

| Status         | Tasks                                                                                                                                                                                                                                |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TO-DO          | <ul style="list-style-type: none"><li>Develop the core functionality</li><li>Designing and Developing...</li><li>Creating Dynamic...</li><li>Preparing the Application</li><li>Testing and Verifying...</li><li>Conclusion</li></ul> |
| IN PROGRESS    | <ul style="list-style-type: none"><li>Pre-requisites</li><li>Define the Architecture...</li><li>Set Up the Development...</li><li>Handle Document...</li></ul>                                                                       |
| TO BE REVIEWED | <ul style="list-style-type: none"><li>Workflow</li><li>Research and Selection...</li><li>Implement the Fas...</li><li>Create and Config...</li></ul>                                                                                 |
| COMPLETED      | <ul style="list-style-type: none"><li>No task visible in the supplied screenshots.</li></ul>                                                                                                                                         |

Visible count: To-Do 6 | In Progress 4 | To Be Reviewed 4 | Completed 0.

## 16. Complete Recording/ Demonstration Steps

- 30. Show the SkillWallet board.
- 31. Introduce LegalEase.
- 32. Show the VS Code project structure.
- 33. Open Terminal 1 and start FastAPI.
- 34. Show the successful Uvicorn startup.
- 35. Open Terminal 2 and start Streamlit.
- 36. Open `http://localhost:8501`.
- 37. Enter the Rental Agreement sample.
- 38. Click Generate Document.
- 39. Show the generated result.
- 40. Edit the document.
- 41. Download the document.
- 42. Open the downloaded file.
- 43. Explain the frontend → backend → AI → output workflow.
- 44. Conclude the demonstration.

## 17. Voice-Over Script

**Introduction:** Hello everyone. Today I am going to demonstrate my project, LegalEase, an AI-powered legal document generator.

**Project structure:** The frontend uses Streamlit and the backend uses FastAPI. The project also contains AI generation and document-formatting components.

**Backend:** I am starting the FastAPI backend using Uvicorn. This backend receives requests and handles the document-generation process.

**Frontend:** Now I am starting the Streamlit frontend. This is the user interface for entering the legal document information.

**Input:** I will enter the document type, parties, terms and effective date.

**Generation:** After clicking Generate Document, Streamlit sends the information to the FastAPI backend. The backend processes the request and returns the generated draft.

**Editing:** The generated document can be reviewed and edited before saving.

**Export:** The documented full implementation provides TXT, DOCX and PDF export options.

**Conclusion:** This completes my LegalEase demonstration. The project combines Streamlit, FastAPI, generative AI and document formatting in one workflow.

## 18. Common Errors and Fixes

| Problem                                       | Fix                                                                                    |
|-----------------------------------------------|----------------------------------------------------------------------------------------|
| <b>pip is not recognized</b>                  | Use <code>.\venv\Scripts\python.exe -m pip</code> instead of <code>pip</code> .        |
| <b>PowerShell Activate.ps1 blocked</b>        | Use the venv Python directly: <code>.\venv\Scripts\python.exe ...</code>               |
| <b>No module named uvicorn</b>                | Install requirements into <code>.venv</code> using its Python executable.              |
| <b>Attribute app not found</b>                | Check that <code>backend/main.py</code> contains <code>app = FastAPI(...)</code> .     |
| <b>ModuleNotFoundError for backend.config</b> | Check the file exists at <code>backend/config.py</code> and run from the project root. |
| <b>Frontend cannot connect</b>                | Start the FastAPI backend first and verify port 8000.                                  |
| <b>Blank/failed frontend</b>                  | Restart Streamlit and verify <code>frontend/app.py</code> has valid code.              |
| <b>Port already in use</b>                    | Stop the old server with <code>Ctrl+C</code> or use another port.                      |

## 19. Future Improvements

- Cloud deployment for frontend and backend.
- Authentication and role-based access.
- Improved protection of sensitive information.
- More legal document templates.
- Jurisdiction-specific configuration.
- More automated tests.
- Enhanced DOCX/PDF formatting.

- Document history and version management.
- Improved prompt controls and output validation.

## 20. Important Usage Note

LegalEase should be presented as a legal document drafting/assistance project. Generated text should be reviewed appropriately before it is relied upon for an actual legal matter. The project documentation does not establish that every generated document is legally valid in every jurisdiction.

## 21. Conclusion

LegalEase demonstrates an end-to-end workflow covering model selection, architecture, environment setup, core functionality, API integration, frontend development, document preview/editing, export, local deployment and testing. The coding examples in this document are intended to show how each project stage maps to actual files and commands in a VS Code implementation.

## 22. Source Basis

This document is based on the LegalEase project documentation supplied in the conversation and the three SkillWallet screenshots. Coding examples have been included to connect the documented activities to a practical VS Code implementation. Where the source screenshots truncate a task title, the missing words have not been guessed.